NOUS RESEARCH · ОФИЦИАЛЬНЫЙ FAQ

30

вопросов новичков о Hermes Agent

Начало работы, модели, память, каналы,
стоимость и наведение порядка --
каждый ответ сверен с документацией

Перевод для @prompt_design

Содержание

Вступление **2**

Начало работы **2**

Модели: локальные и облачные **4**

Память **6**

Каналы и работа нон-стоп **6**

Стоимость **7**

Рабочие приемы **8**

Скиллы и развитие **8**

Напоследок **9**

Вступление

Перевод FAQ команды Nous Research для @prompt_design. Оригинальные ответы сверены с документацией Hermes Agent.

Это справочник по 30 вопросам, которые новички задают о Hermes Agent чаще всего. Команда Nous Research собрала их в сообществе на Reddit, X и Discord.

Каждый ответ сверен с актуальной документацией. Если сомневаетесь, последнее слово за ней, и вы всегда можете попросить агента свериться.

Вопросы сгруппированы по темам, от начала работы и моделей до памяти (memory), каналов, стоимости, рабочих приемов и скиллов (skills). Каждый вопрос можно читать отдельно, не оглядываясь на остальные.

Начало работы

1. «Я в сессии? В проекте? Что значат все эти слова?»

Четыре слова, разбираем один раз.

- Сессия (session) – один разговор. У агента их много, и открыть сессию можно с любого устройства. Начали в приложении, продолжили в терминале.
- Проект (project) – рабочее пространство. Папки, в которых работает агент, собранные под одним именем.
- Профиль (profile) – полностью отдельный агент со своими настройками, памятью, скиллами и историей.
- Бот (bot) – профиль с именем и аватаром в настольном приложении.

Сессии и проекты упорядочивают вашу работу. Профили и боты – это агенты, которые ее выполняют.

И еще одно слово бонусом. Гейтвей (gateway) – процесс, который работает постоянно и соединяет агента с Telegram, Discord, WhatsApp и другими мессенджерами.

2. «Какой файл – мозг моего агента?»

Четыре файла, у каждого своя задача.

- SOUL.md – кто ваш агент. Лежит только в `~/.hermes/`.
- AGENTS.md – как устроен конкретный проект. Лежит в папке проекта. Внутри git-репозитория Hermes объединяет всю цепочку таких файлов, от корня репозитория до папки, в которой вы работаете.
- MEMORY.md – что агент уже узнал. Этот файл он пишет сам.
- USER.md – кто вы для него. Этот файл тоже пишет агент.

Первые два пишете вы. Последние два – он.

3. «С какой модели начать?»

Если совсем не знаете, с чего начать, берите бесплатную модель на Nous Portal, прямо сегодня. `hermes setup --portal` одной командой войдет в аккаунт и настроит провайдера, а в списке моделей вы увидите короткую подборку для каждого провайдера, а не свалку из трехсот названий.

Выбор модели ни к чему не обязывает. Сменить ее потом – одна команда (`/model` в любом чате). Только учтите, что смена модели посреди разговора сбрасывает кэш промпта (prompt cache), и следующее сообщение заново прочитает весь разговор по полной стоимости. Меняйте модель в самом начале или уже в новой сессии.

4. «Я открыл его… и что дальше?»

Лучший ответ в обсуждении повторяли сами пользователи, почти слово в слово. Возьмите одну настоящую задачу из своего дня и поручите ее агенту. Одна надоевшая мелочь сегодня. Следующая – завтра.

Вот как это выглядело в ответах. У одного агент держит в порядке финансы. Другой за два дня собрал бота, который пишет людям на Reddit. У третьего все началось с того, что агент появился в Slack, и с тех пор он пользуется им каждый день. Агент приживался у тех, кто доверил ему настоящее дело.

5. «Какие скиллы поставить в первую очередь?»

Скорее всего, никакие. Память (memory), поиск в интернете, управление браузером, регулярные задачи (recurring jobs) и субагенты (subagents) встроены в агента, это не скиллы (skills). Ставьте скилл, когда столкнетесь с задачей, которую встроенные возможности не закрывают. Не раньше.

6. «Мой Hermes крутится на сервере. Как пользоваться им с ноутбука?»

Обычно агент живет на машине, которая никогда не выключается, а вы работаете с ноутбука, который засыпает. Соединить их можно в три шага.

1. На сервере держите запущенным `hermes serve` (через systemd, tmux или любым другим способом, который переживает выход из системы).
2. В настольном приложении откройте Settings -> Gateways -> Remote gateway и введите адрес сервера. Гейтвей (gateway) здесь – тот постоянно работающий процесс на сервере, к которому подключается приложение.
3. Войдите. Если сервер слушает адрес, доступный снаружи, проверка входа включается автоматически.

Одно правило безопасности. Вход по логину и паролю подходит только для доверенной сети или VPN вроде Tailscale. Если сервер доступен из открытого интернета, используйте вариант «Sign in with Nous Research».

А если вам нужно не подключиться к агенту, а перенести его, есть два пути. `hermes backup` и `hermes import` переносят все целиком, `hermes profile export` переносит одного агента, но без учетных данных.

7. «Почему простое сообщение стоило так дорого?»

В начале каждой сессии (session) агент загружает в контекст один и тот же набор данных, о чем бы вы ни спросили. Это системный промпт (system prompt), описания инструментов, каталог скиллов и память. Команда `hermes prompt-size` показывает размер этого набора с точностью до байта, не обращаясь к модели. Дальше можно отключить наборы инструментов, которыми вы не пользуетесь (`hermes tools`), удалить скиллы, которые никогда не запускаете, и не раздувать AGENTS.md.

8. «Он запомнил… но пока этого не видит?»

Память записывается на диск сразу, а в системный промпт подгружается только при старте сессии. Факт, сохраненный посреди разговора, никуда не денется и появится в следующей сессии. Если он нужен прямо сейчас, начните новую.

9. «Когда мне нужен второй агент?»

Позже, чем кажется. Пользователи в обсуждении советуют одно и то же. Один основной агент, с которым вы разговариваете, и второй профиль (profile) только тогда, когда почувствуете, что разделение действительно нужно, например на рабочего и личного. Оркестрация (канбан-доски, маршрутизация по профилям) – самая сложная часть продукта, к ней приходят в последнюю очередь.

10. «А если я запутаюсь?»

Спросите агента. Несколько человек в обсуждении написали примерно одно и то же, «спросите своего агента, как тут все устроено», и они правы. Он умеет читать свою документацию, объяснять свои настройки и чинить собственную установку (`hermes doctor` для диагностики, `hermes doctor --fix` для автоматического исправления).

Модели: локальные и облачные

11. «Какое железо нужно, чтобы гонять Hermes полностью локально?»

В память машины должны поместиться веса модели вместе с кэшем контекста, и еще Hermes нужен запас, чтобы было где думать. Для агентской работы нужно окно контекста (context window) на 64 000 токенов, потому что в него должны уместиться системный промпт (system prompt), схемы инструментов и вся ваша переписка.

Если нужны ориентиры, то GPU на 24 ГБ спокойно тянет квантованную модель примерно на 27B параметров с полным окном в 64 000 токенов. На 12-16 ГБ хорошо работают модели поменьше. Если памяти еще меньше, смотрите в сторону моделей по API или гибридного варианта, когда агент работает в облаке, а при необходимости управляет вашим локальным компьютером.

12. «Локальная модель говорит, что делает, но ничего не происходит. Почему?»

Модель пересказывает вызовы инструментов словами вместо того, чтобы выполнять их по-настоящему. Причин обычно две.

Первая в сервере. Ему нужен флаг, который включает вызов инструментов. llama.cpp запускают с `--jinja`, vLLM с `--enable-auto-tool-choice` и `--tool-call-parser`, у SGLang тоже есть свой флаг парсера. Без этих флагов модель выводит вызовы инструментов обычным текстом.

Вторая в самой модели. Прежде чем на ней остановиться, проверьте в карточке модели, поддерживает ли она вызов инструментов (tool calling или function calling). Если в карточке про инструменты ни слова, никакие флаги не спасут.

13. «Почему локальная модель думает над первым сообщением несколько минут?»

Все, что агент получает на старте, то есть системный промпт, схемы инструментов, список скиллов (skills) и память (memory), весит десятки тысяч токенов, и локальная машина должна пропустить все это через себя, прежде чем написать первое слово. Скорость такого чтения контекста (prefill) и скорость генерации текста – разные характеристики, и потребительские GPU справляются со второй гораздо лучше, чем с первой.

Так дорого обходится только первый ответ. Дальше прочитанное попадает в кэш промпта (prompt cache), и следующие сообщения обрабатываются быстро. Команда `hermes prompt-size` показывает точный объем этого стартового набора, а если отключить неиспользуемые наборы инструментов, он станет меньше.

14. «Локальный агент забывает все посреди задачи. Что не так?»

Почти всегда причина в том, что окно контекста (context window) меньше, чем вы думаете. Серверы по умолчанию не дают модели все окно, на которое она рассчитана. У Ollama оно зависит от объема VRAM и может быть всего 4096 токенов, а llama.cpp делит окно между слотами, если включить параллельные запросы.

Выставьте серверу 64 000 токенов или больше (`OLLAMA_CONTEXT_LENGTH=64000 ollama serve` для Ollama, `-c 64000` для llama-server) и пропишите то же значение в конфиге Hermes, `context_length: 64000`. Проверить можно через `ollama ps`, в колонке CONTEXT должно стоять то число, которое вы задали.

15. «Может агент гонять дешевую модель для фоновой работы и сильную для чата?»

Да. Субагенты (subagents), то есть помощники, которых агент запускает для параллельных и фоновых задач, по умолчанию работают на той же модели, что и основной диалог. Но через `delegation.model` им можно назначить любую модель подешевле, у любого провайдера. Основной разговор остается на сильной модели и сохраняет свой кэш промпта (prompt cache), а массовая работа идет по ценам дешевой модели.

Тот же принцип действует и дальше. У регулярных задач (cron jobs) и служебных операций вроде сжатия контекста и генерации заголовков модель тоже задается отдельно.

16. «Hermes работает с OpenRouter, моим llama.cpp, LM Studio?..»

Да. Любой сервер с OpenAI-совместимым API подключается одним блоком в конфиге. Указываете провайдера `custom` и `base_url` своего сервера, и готово. Этот формат поддерживают Ollama, vLLM, сервер llama.cpp, SGLang, LM Studio и LocalAI. А у OpenRouter и еще трех десятков облачных провайдеров есть свои готовые интеграции, которые настраиваются через `hermes model`.

Память

17. «Как поправить то, что агент запомнил неправильно?»

Скажите ему об этом обычными словами, например «Факт про X неверный, запомни вместо него Y». Инструмент памяти (memory) находит записи по вашему описанию и сам правит или удаляет их, искать файл или знать ID не нужно. Можно зайти и с другой стороны, спросить «Что у тебя в памяти про X?» и убрать лишнее из списка.

Память по замыслу небольшая и выверенная (пара тысяч символов, примерно 8-15 записей), так что ошибочная запись там редкость, и исправить ее несложно.

18. «Могут два агента делить одну память?»

Не через общие файлы. Оба агента пишут в память автоматически, и если два автора работают в одной домашней папке, их записи накладываются друг на друга, пока не получится что-то, чего не задумывал ни один из них (и вы тоже). Поэтому у каждого агента свой профиль (profile).

Когда агентам действительно нужны общие знания, есть внешние провайдеры памяти. Это общее хранилище, к которому оба агента обращаются через штатный инструмент, а рабочая память у каждого остается своя.

19. «Чем память отличается от истории сессий?»

Память – то, что агент берет с собой в каждый разговор. Предпочтения, факты об окружении, усвоенные уроки. Она небольшая, выверенная и загружается всегда. История сессий (session history) – все, что вы когда-либо обсуждали. Она полная, и по ней можно искать (для этого у агента есть инструмент `session_search`).

Если коротко, память отвечает на вопрос «что агент должен знать без напоминаний?», а история – на вопрос «что мы об этом говорили?». Агент сам пользуется и тем и другим, вам почти не приходится думать об этом разделении.

Каналы и работа нон-стоп

20. «Как пользоваться Hermes с телефона?»

Подключите мессенджер. Telegram, WhatsApp, Discord, Slack, Signal и другие уже встроены. Запустите гейтвей (gateway), то есть постоянно работающий процесс, который соединяет агента с мессенджерами, привяжите свой аккаунт, и агент со всеми своими возможностями будет под рукой в чате, где бы вы ни были. Он выполняет задачи, присылает файлы и отвечает голосовыми сообщениями. Причем агент везде один и тот же. Память (memory), сессии (sessions) и скиллы (skills) принадлежат профилю (profile), а не устройству, поэтому разговор, начатый на телефоне, продолжится на ноутбуке.

21. «Может он работать, пока я сплю?»

Да, именно для этого есть регулярные задачи (recurring jobs). Настраиваются они обычной просьбой, например «Каждое утро в 7 проверяй мои ленты и присылай сводку в

Telegram». Агент сам поставит задачу в расписание. При каждом запуске он просыпается в новой сессии, выполняет работу и присылает результат туда, куда вы просили.

Утренние сводки, отслеживание изменений на странице, еженедельные отчеты, ночное резервное копирование – принцип везде один и тот же.

22. «Мне нужен сервер? Мой ноутбук засыпает»

Чтобы задачи по расписанию срабатывали ночью, какая-то машина должна в это время работать. Обычно выбирают один из трех вариантов, от самого хлопотного к самому простому. Это небольшой VPS, домашний компьютер, который не выключают (старого ноутбука или мини-ПК вполне хватит), или Hermes Cloud, где о постоянной работе позаботятся за вас.

23. «Могу я разговаривать с агентом голосом?»

Да, причем способов больше, чем многие ожидают.

- В настольном приложении и в командной строке есть голосовой режим (voice mode). Зажимаете клавишу, говорите, агент отвечает вслух.
- Можно и без рук. Фраза активации (wake word) вроде «привет, Hermes» или любая другая на ваш выбор запускает голосовую сессию. По умолчанию эта функция выключена.
- В Telegram и Discord можно отправить голосовое сообщение, агент расшифрует его и ответит. При желании он и ответ озвучит.
- В Discord возможностей еще больше. Бот (bot) заходит в голосовой канал и ведет живой разговор со всеми, кто в нем находится.

Распознавание речи может работать полностью локально, без единого ключа API, а с подпиской Nous Portal все голосовые функции доступны сразу, без дополнительной настройки.

Стоимость

24. «Какой самый дешевый разумный вариант для новичка?»

Начинайте с бесплатного. В Nous Portal есть бесплатные модели (наберите FREE в поиске по каталогу, там актуальный список), а `hermes setup --portal` подключает все одной командой. Так вы получите рабочего агента и не будете считать каждый токен, пока разбираетесь, зачем он вам вообще нужен.

Дальше две привычки. Держите AGENTS.md и набор инструментов компактными, тогда каждой сессии (session) на старте достается меньше вводного текста. А если начнете поручать задачи фоновым агентам, закрепите за ними модель подешевле через `delegation.model`, основной чат при этом останется на сильной.

25. «Как посмотреть, сколько я трачу?»

Команда `/usage` в любом чате. Она показывает расход токенов, примерную разбивку по стоимости, длительность сессии, а если провайдер отдает такие данные, то и текущий остаток квоты или кредитов прямо из его API.

Расход виден и в Nous Portal.

Разобраться, из чего складывается расход, помогает `hermes prompt-size` (работает офлайн и бесплатно). Команда показывает, сколько токенов уходит на старте каждой сессии и на что именно, так что вы еще до чистки поймете, сколько сэкономите.

Рабочие приемы

26. «Как дать ему работать с кодом, не трогая основную ветку?»

Через рабочие копии (worktree). Запустите `hermes --worktree`, и сессия пойдет в отдельной рабочей копии git на собственной ветке, а ваш рабочий каталог останется чистым, пока агент работает. Нужно делать две вещи в одном репозитории параллельно? Тот же прием, повторенный дважды. Два агента, две рабочие копии.

Вдобавок Hermes по ходу работы сохраняет контрольные точки изменений в файлах. `/rollback` выводит их список и откатывает к выбранной, а `/diff session` собирает все, что изменила сессия, в одном месте.

27. «Может он проверить свою работу? Могут два агента работать параллельно?»

И то и другое. `/review` запускает отдельного проверяющего агента со свежим взглядом и полным набором инструментов. Он открывает PR, читает diff и запускает код, а не судит по краткому пересказу. Поскольку это отдельный агент со своим контекстом, у него нет допущений, которые накопила сессия, писавшая код.

Параллельная работа устроена по тому же принципу. Агент запускает субагентов (subagents) под независимые задачи, по умолчанию до трех одновременно, каждого в отдельном контексте. Вместе с рабочими копиями так получается несколько агентов на одном репозитории сразу, каждый на своей ветке.

Скиллы и развитие

28. «Скиллы, плагины, MCP: в чем разница и что мне нужно?»

- Скилл (skill) – это знание. Документ, который учит агента определенному рабочему процессу (как вы оформляете расходы, как выкатываете обновления). Скиллы подгружаются по мере надобности, и каждый установленный скилл можно вызвать командой через `/`.
- Плагин (plugin) расширяет само настольное приложение Hermes, добавляет новые панели и элементы интерфейса.
- MCP подключает внешние серверы инструментов, то есть сторонние сервисы, которые общаются по единому протоколу.

На старте не нужно ничего из этого, встроенных возможностей хватает на очень многое. Первым обычно понадобится скилл, причем нередко его пишет сам агент. Проведите его по рабочему процессу один раз, а затем `/learn` превратит этот опыт в скилл, который останется с ним насовсем.

29. «Установленные скиллы жгут токены, даже когда не используются?»

Скиллы работают по принципу постепенного раскрытия (progressive disclosure). По умолчанию в контекст попадает только одна строка из индекса, а полный документ подгружается, лишь когда задача этого требует. Так что установленный, но неиспользуемый скилл за сессию (session) почти ничего не стоит.

Почти ничего, но не ноль. Строка в индексе тоже занимает место, и сотня скиллов, которыми вы никогда не пользуетесь, все равно раздувает системный промпт (system prompt). В выводе `hermes prompt-size` есть отдельная строка про скиллы. Удаляйте те, которыми точно не будете пользоваться.

30. «Мои скиллы и память заросли за несколько месяцев. Как навести порядок?»

За скиллами следит встроенный смотритель (groundskeeper). Это фоновый процесс, который помечает скилл как устаревший через 30 дней без использования и через 90 дней отправляет в архив. Именно в архив, а не в корзину. Все можно восстановить из папки архива, а закрепленные скиллы он не трогает вовсе.

С памятью (memory) проще всего попросить агента напрямую. «Пересмотри свою память, объедини дубли, убери устаревшее». Хранилище по замыслу небольшое и выверенное, так что уборка проходит легко. Если хочется убраться основательнее, попросите его так же пересмотреть свои скиллы. Наведение порядка – такая же задача, как любая другая.

Напоследок

Вот и все 30. Более сложные темы, которые вы поднимали, тоже не потерялись. Простой канбан, устройство памяти, неудобства удаленной настройки – все это у команды в работе.

Мы, разумеется, читаем каждый ответ. Если вашего вопроса нет в списке, напишите нам. В документации все, о чем говорилось выше, разобрано подробнее, а в Discord Nous Research полно людей, которые сами совсем недавно делали первые шаги. Спрашивайте дальше. Этот текст появился потому, что вы спрашивали.